OCTOBER 2026 EDITION

ROS 2 Security and Threat Modeling

Trust boundaries, access control, and a practical SROS 2 lab

Juan Carlos Nevarez · Gh0stlyKn1ght

Security review, technical revision, and additional material
@Gh0stlyKn1ght

Adapted from the ROS 2 design documents.
Original authors are credited in each chapter.

Contents

1. Scope and evidence
2. ROS 2 DDS-Security integration
3. ROS 2 Access Control Policies
4. ROS 2 Security Enclaves
5. ROS 2 Robotic Systems Threat Model
6. Full teaching-system threat records
7. Libraries and technology choices
8. Bounded SROS 2 teaching lab
9. Recorded lab results
10. Complete example policy
11. Credits and license notices

Scope and evidence

This handbook explains DDS security, least-privilege policies, enclave boundaries, and threat modeling through a non-actuating talker/listener system. The practical baseline is ROS 2 Jazzy on Ubuntu 24.04 with Fast DDS. The four chapters lead into a structured threat register, technology choices, a reproducible lab, and its recorded results.

The recorded software run passed six cases; the independently generated authority case remains inconclusive. These observations apply to the named versions and settings. They do not establish host isolation, actuator safety, or the security of a physical robot. Use the results chapter to distinguish observed behavior from controls that still require testing.

ROS 2 DDS-Security integration

Original authors: Kyle Fazzari

2026 revision, security review, and additional material: Juan Carlos Nevarez · Gh0stlyKn1ght

Originally written: 2019-07 · Revised: 2026-10-20

Original article.

Goal and prerequisites

Explain which threats DDS security can address, configure enforcement deliberately, and identify controls that belong to the operating system or application. You should understand ROS nodes, topics, services, contexts, processes, and basic public-key certificates. Use the access-control chapter for authorization and the enclaves chapter for credential placement.

Practical question: If a participant presents a trusted certificate, should it be allowed to publish an actuator command? Authentication establishes an identity; permissions decide which operations it may perform, and the application still decides whether a permitted command is valid and safe.

Scope and platform choice

This chapter describes **DDS-based ROS 2 middleware with DDS security support**. ROS 2 is middleware running on an operating system; it does not replace host access control, patching, network policy, or an independent safety system.

Use one ROS distribution and one DDS implementation for the first exercise. The accompanying lab targets **Ubuntu 24.04, ROS 2 Jazzy, and `rmw_fastrtps_cpp`**, with exact installed package versions captured by the operator. This configuration was exercised in the isolated Ubuntu userspace described in the runtime observations: six ROS cases passed and the separate-authority case remained inconclusive.

For a new deployment, evaluate **Lyrical on Ubuntu 26.04** when hardware drivers and application packages support it. The official release table lists Lyrical support through May 2031 and Jazzy through May 2029. Crystal is end of life and belongs only in the historical case studies. Do not mix distribution-specific installation instructions or assume an existing robot can be upgraded without vendor validation. See the official release table and Lyrical platform support.

`rmw_zenoh_cpp` has a different configuration and trust model. Its security tooling can reuse SROS 2 policies and key material, but applies them through generated Zenoh session/router configurations. DDS governance files and these environment variables are not a universal recipe for every RMW. See the Zenoh security tool documentation.

DDS security properties

The DDS Security architecture defines authentication, access-control, cryptographic, logging, and data-tagging plugin interfaces. The first three establish the main identity, authorization, and message-protection functions used here. Audit logging availability and data-tagging behavior depend on the vendor and build; do not infer that a usable audit trail exists because encryption is enabled.

Property	What it establishes	What remains outside it
Authentication	A participant can demonstrate an identity trusted by the configured authority.	Whether the holder is uncompromised or its command is appropriate.
Access control	Signed governance and permissions authorize configured operations.	Host file access, process isolation, or command semantics.
Cryptographic protection	Configured DDS traffic can receive integrity/authenticity and confidentiality protection.	Data after decryption, all metadata, non-DDS links, or availability under overload.
Logging	Supported, configured security events can aid investigation.	Complete forensic evidence, retention policy, or tamper resistance by default.

The builtin plugin names commonly used in DDS security are `DDS:Auth:PKI-DH`, `DDS:Access:Permissions`, and `DDS:Crypto:AES-GCM-GMAC`. The access-control name is plural **Permissions**. Use your vendor's documentation for supported algorithms, protection settings, and plugin-loading requirements; do not design a new cryptographic scheme for a tutorial.

The original design cited DDS Security 1.1. OMG now publishes version 1.2. That does not prove a particular ROS package implements every 1.2 feature. Record the standard basis supported by the actual middleware and avoid copying old section numbers into a new compatibility claim.

Interoperability is a test obligation

The official Jazzy security setup source says secure cross-vendor communication is not supported. Keep the same vendor for this lab. If a project requires a different combination, verify its support statement and test the exact versions, governance, permissions, discovery behavior, and recovery paths. Using standard plugin names alone is not proof that the combination works.

Governance, permissions, and identity

Identity certificates bind a participant's public key to a subject trusted by the identity authority. **Governance** specifies domain/topic protection and admission behavior. **Permissions** grant operations to a certificate subject for domains and topics during a validity interval. Permissions need signing; editing XML without producing a matching signed permissions file does not update the runtime policy.

Review governance explicitly: unauthenticated-participant admission, join access control, discovery protection, metadata protection, and payload protection. A participant joining a domain is not the same operation as publishing or subscribing to a particular topic. Inspect generated governance and permissions rather than assuming a tool's example defaults meet your deployment requirements.

Neither the ROS node name, its namespace, nor `ROS_DOMAIN_ID` is an authenticated security identity. A domain ID separates discovery domains by convention; it is not an access-control boundary. An authorized process can still misuse the permissions granted to its identity.

Configure enforcement deliberately

For a DDS RMW, set the environment in **every participating terminal or service**, and select an absolute enclave path separately:

```
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export ROS_SECURITY_KEYSTORE="$HOME/ros2-security-lab/keystore"
export ROS_SECURITY_ENABLE=true
export ROS_SECURITY_STRATEGY=Enforce
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/talker
```

`ROS_SECURITY_KEYSTORE` points to the **keystore root containing `enclaves/`**. The expected runtime files normally include `cert.pem`, `key.pem`, `identity_ca.cert.pem`, `permissions_ca.cert.pem`, `governance.p7s`, and `permissions.p7s`. The process needs read access only to its required material.

`ROS_SECURITY_ENABLE=true` enables the DDS security path; `ROS_SECURITY_STRATEGY=Enforce` requests failure when required security configuration cannot be established. Without the deliberate enforcement setting, permissive fallback can permit unsecured startup when material is missing. Treat a successful process launch as insufficient evidence: test authorized delivery and rejection separately.

`ROS_SECURITY_ENCLAVE_OVERRIDE` overrides the selected enclave path. It does not replace the keystore root. Remove an unexpected inherited override before a controlled exercise:

```
unset ROS_SECURITY_ENCLAVE_OVERRIDE
```

A CLI or daemon is another participant and needs an appropriate identity and policy. Use a dedicated inspection identity in a deployed system; copying a production participant's keys into an administrator's shell enlarges its trust boundary.

Credential lifecycle and host controls

Keep signing-authority private keys away from runtime robots. An enclave private key is still a secret even though it is inside the `enclaves/` directory. An SROS 2 enclave is a credential/policy grouping, not a hardware protected execution environment.

Define provisioning, expiry, renewal, compromise response, backup, policy signing, clock requirements, and recovery ownership. For revocation or hardware-backed keys, obtain a vendor/version-specific implementation and test it. Do not assume an online revocation service or hardware key integration is available in every DDS security stack. Keep software-update signing separate from participant identity and permission signing.

Run services with separate OS identities where meaningful, restrict file access and capabilities, patch supported packages, and protect remote administration. Middleware authentication does not stop a privileged local attacker from reading accessible keys or replacing an application binary. A firewall, VPN, or container can add boundaries; none substitutes for a tested participant policy.

Bounded exercise and evidence

Use the included lab on an isolated, non-actuating workstation. It supplies a policy, an offline schema/transform check, a setup script that refuses to overwrite a keystore, and expected positive/negative cases. The offline checks validate generated permission structure and scope; the ROS communication cases must be run on the selected ROS host.

Produce a version manifest, a diagram, the reviewed governance and signed permissions hashes, and observations for authorized delivery, a missing enclave, an untrusted identity, and a forbidden topic. A silent listener alone is not evidence of authorization working; rule out networking, type, and QoS failures using the authorized control case and logs.

Explain back: Why can an authenticated participant be forbidden from publishing `/cmd_vel`? Why can a stolen authorized key still be dangerous when traffic is encrypted?

Common misconceptions: enabling security is not the same as enforcing a restrictive policy; encrypting a channel does not validate application commands; distinct node names do not isolate shared process memory.

Limit: the recorded software tests do not establish physical-robot safety, host isolation, revocation behavior, or packet-level confidentiality. The separate-authority result still needs stronger rejection evidence. Next, define the exact publish/subscribe/service/action privileges in the access-control chapter.

Sources

- SROS 2 project, using distribution branches and pinned versions.
- Fast DDS security.
- Cyclone DDS security.
- DDS Security 1.1 and 1.2.

ROS 2 Access Control Policies

Original authors: Ruffin White, Kyle Fazzari

2026 revision, security review, and additional material: Juan Carlos Nevarez · Gh0stlyKn1ght

Originally written: 2019-08 · Revised: 2026-10-20

Original article.

Goal and prerequisites

Turn a data-flow requirement into a reviewable policy and explain the difference between structural validation and runtime enforcement. Read the DDS security chapter first. You need basic XML knowledge and familiarity with ROS topics, services, actions, namespaces, and remapping.

Practical question: Can a listener identity also publish the data it is meant to read? Our example grants subscription to the application topic only; it separately grants the infrastructure operations needed by the chosen nodes.

Access-control model

Apply least privilege to the actual authenticated principal, which in a DDS deployment is associated with a participant and its enclave material. A ROS node name is useful for policy authoring but is not, by itself, a cryptographic principal. Nodes sharing a context share its participant identity and effective permissions; processes sharing memory cannot be treated as mutually isolated merely because policy profiles differ.

A mandatory policy is enforced outside the application's discretionary choices, but a compromised enforcement process or privileged host can still defeat the assumed boundary. Document which system component is trusted to enforce each rule.

Separate these representations:

1. **Requirements:** intended data flows and operations, including denied flows.
2. **SROS 2 policy:** named enclaves and ROS-oriented profiles.
3. **DDS permissions:** generated transport names, certificate subject, domains, and validity.
4. **Signed runtime material:** permissions and governance accepted by the middleware.
5. **Evidence:** actual authorized and rejected behavior on the recorded implementation.

A schema-valid policy can still be too broad, mapped incorrectly, expired, signed by the wrong authority, or deployed to the wrong process.

Policy format and version

The included example uses SROS 2 policy **0.2.0**. It is checked against the pinned Jazzy schema in the offline vendor bundle. The XML below is intentionally small; the runnable lab policy additionally accounts for logging, parameter infrastructure, and time-related behavior used by the example nodes. Do not deploy this abbreviated fragment as a complete node policy.

```
<policy version="0.2.0">
  <enclaves>
    <enclave path="/security_lab/talker">
      <profiles>
        <profile ns="/" node="talker">
          <topics publish="ALLOW">
            <topic>/chatter</topic>
          </topics>
        </profile>
      </profiles>
    </enclave>
    <enclave path="/security_lab/listener">
      <profiles>
        <profile ns="/" node="listener">
          <topics subscribe="ALLOW">
            <topic>/chatter</topic>
          </topics>
        </profile>
      </profiles>
    </enclave>
  </enclaves>
</policy>
```

An `enclave` selects credential scope; a `profile` describes a node name and namespace for expressing resource names. A profile does not create an OS sandbox. The `topics`, `services`, and `actions` elements specify direction-specific operations. Confirm the exact schema and transformations shipped with the chosen SROS 2 package before adopting an example from another distribution.

Direction and generated resources

Requirement	Policy operation	Review question
Publish or read data	`publish` / `subscribe`	Can this principal also write a topic it should only read?
Call or provide a service	`request` / `reply`	Are both request and response transport directions correct?
Use or provide an action	`call` / `execute`	Are goal, result, cancellation, feedback and status covered?

A ROS service or action expands into more than one DDS resource. Review the generated permissions as well as the friendly policy. Changing the selected RMW, namespace, node name, or a remapped topic can change those resources.

Prefer explicit names for the first lab. A wildcard is a deliberate grant over a set of names, including future matching names; record why that set is acceptable. Do not grant every resource under a prefix merely to eliminate startup errors.

XML and tooling trust boundary

The original design describes XInclude and XSLT composition. These features can read additional resources, so accept policies and transforms only from reviewed sources. This edition's example has **no XInclude** and no DTD/entity declarations. The offline checker rejects those constructs, restricts schema resolution to an explicit local file, disables network resolution, and denies transform file/network access.

The pinned `lxml`-based check validates both the policy and generated DDS XML. It also checks actual topic direction, default denial, certificate subjects, domain scope, and absence of unrelated application resources. This is a useful local check, not a replacement DDS authorization engine or a generic validator for arbitrary externally supplied policy bundles.

Generate and inspect permissions

On the selected ROS host, the lab uses `ros2 security create_keystore`, `create_enclave`, and `create_permission`. The setup script applies the reviewed policy to each enclave rather than leaving the broadly useful demo permissions in place. Use the installed command's help for supported options and verify generated validity dates. The offline reference transform's default dates are not an operational certificate-rotation policy.

Before running a participant:

- Check the certificate subject matches the permission grant.
- Check the domain includes the intended lab domain and no unintended domains.
- Inspect allow and deny rules, especially wildcard expressions and infrastructure topics.
- Inspect governance admission and protection settings.
- Verify signatures and time validity with the chosen tools.
- Ensure the signed file actually deployed is the one reviewed.

The official access-control tutorial demonstrates policy signing and remap rejection. Keep signing keys on the provisioning system; the robot ordinarily needs runtime artifacts rather than authority private keys.

Policy changes and composition

Regenerate and sign permissions after a relevant name or privilege change. A different enclave path does not automatically make a profile's topic names match a new namespace. Record the effective namespace and remappings from launch configuration. If components share an enclave, review the union of their permissions as the actual attack surface.

Treat a policy inferred from a running graph as a starting observation. It can miss rarely used service calls, startup paths, faults, and dynamic operations, and can learn unnecessary or malicious behavior already present in the observed system. Review it against intended behavior and explicit negative requirements.

Exercise, misconceptions, and next step

Run the offline checker in the lab directory. Inspect the generated listener grant and identify which topics it can publish for infrastructure and which application topic it can only read. Then, on a ROS host, compare authorized `/chatter` delivery with a remap to `/not_chatter` using the same restricted identity. Record initialization rejection or forbidden endpoint behavior as observed; do not invent a uniform error message across vendors.

Artifact: reviewed policy, generated permissions, version manifest, and a result table with logs.

Explain back: Why does “valid XML” not imply least privilege? Which permissions would two components effectively have after sharing one enclave?

Misconceptions: authorization is not achieved by encryption alone; node names are not protected principals; default denial does not make a broad allow rule narrow; a domain ID is not a firewall.

Limit: offline checks and the recorded Jazzy/Fast DDS run establish the specific observed grant/startup behavior. The separate-authority rejection mechanism remains inconclusive; other deployments need their own run. Next, choose enclave and process boundaries before applying this policy to a robot.

Sources and schema

- Pinned SROS 2 policy schema.
- Pinned permission transform.
- lxml document resolution and XSLT access controls.
- Original schema retained in this repository; historical reference only, not a promise about every installed distribution.

ROS 2 Security Enclaves

Original authors: Ruffin White, Mikael Arguedas

2026 revision, security review, and additional material: Juan Carlos Nevarez · Gh0stlyKn1ght

Originally written: 2020-05 · Revised: 2026-10-20

Original article.

Goal and prerequisites

Choose an enclave scope that reflects a real trust boundary and deploy its runtime artifacts without distributing authority private keys. Read the DDS security and access-control chapters first.

Practical question: Do two different enclave paths inside one process isolate a trusted actuator component from untrusted code? They do not create a memory boundary. If one component can read another's memory or credentials, distinct participant identities cannot repair that shared-process trust assumption.

Names, contexts, participants, and processes

Term	Role	Security implication
Node name / namespace	Names ROS graph resources.	Useful for organization; not an authenticated identity.
Context	ROS client-library execution scope.	In the DDS mapping discussed here, associated with a participant and its credentials.
DDS participant	Middleware communication entity.	Uses an identity and effective permissions.
SROS 2 enclave	Path-selected set of security artifacts and policy scope.	Credential grouping; not a hardware enclave or OS sandbox.
OS process / account	Host execution and access-control boundary.	Governs shared memory and who can read files or inspect another process.

A process can contain multiple nodes and contexts. Describe the actual mapping for the selected RMW/version instead of assuming “one node, one certificate” or “one process always means one context.” All nodes using the same participant identity receive that identity's effective permission set. Shared OS privileges and shared memory can be an even wider boundary.

Keystore root and artifact layout

Set `ROS_SECURITY_KEYSTORE` to the root **containing** `enclaves`, not to the `enclaves` directory itself. The latter instruction in the original article was incorrect. A representative provisioning layout is:

```
keystore/
  public/                # authority certificates
  private/               # authority signing keys; provisioning only
  enclaves/
    security_lab/
      talker/
        cert.pem
        key.pem            # this identity's private key
        identity_ca.cert.pem
        permissions_ca.cert.pem
        governance.p7s
        permissions.p7s
      listener/
        ...
```

Public authority certificates establish trust; they are not secret signing keys. The `private/` authority directory stays off the runtime robot. The runtime enclave's `key.pem` is also private material and needs restrictive access. Do not mistake the directory name `enclaves` for permission to distribute its entire contents.

A runtime deployment may contain a reduced keystore tree with only the required enclave path and certificates. If generated artifacts contain symlinks to `public/`, preserve the required target certificates or materialize the required copies. Check file readability as the actual service account before starting the process. Copying a broken symlink is not a successful credential deployment.

Select an enclave explicitly

```
export ROS_SECURITY_KEYSTORE="$HOME/ros2-security-lab/keystore"
export ROS_SECURITY_ENABLE=true
export ROS_SECURITY_STRATEGY=Enforce
unset ROS_SECURITY_ENCLAVE_OVERRIDE
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/talker
```

The path `/security_lab/talker` identifies the enclave within the keystore. It does not automatically change the ROS namespace or make the executable's node name an authenticated identity. `ROS_SECURITY_ENCLAVE_OVERRIDE` can override the selected path and should be tracked in service/launch configuration.

Launch configuration

Use explicit enclave arguments in a reviewed launch file, for example:

```
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([
        Node(
            package='demo_nodes_cpp',
            executable='talker',
            arguments=['--ros-args', '--enclave', '/security_lab/talker'],
            output='screen',
        ),
    ])
```

The surrounding service or environment supplies the keystore root, enable flag, enforcement strategy, and selected RMW. The snippet is illustrative and has not been executed here. Avoid hidden enclave overrides in included launch files. The original proposals for automatically pushing relative enclave paths with namespaces are design history, not behavior this edition promises is implemented.

Choose the smallest practical trust scope

Start with separate processes and enclave material for components with different trust or command authority. Use separate OS identities and restrictive key-file permissions where the host design supports them. A policy can describe a union of privileges for deliberately composed nodes, but it should not conceal that union.

Per-robot or per-fleet identities can simplify provisioning and also enlarge the impact of key theft. A shared identity reduces attribution and complicates replacement of one compromised robot. Record the operational tradeoff rather than treating a larger enclave scope as a free simplification.

Containers share a host kernel unless stronger isolation is deliberately supplied. Host mounts, shared IPC, excessive capabilities, and shared credentials can cross the boundary the diagram appears to draw. A container name or a separate enclave directory is not evidence of isolation.

Hardware roots of trust and trusted execution environments are additional platform capabilities. Verify compatibility with the middleware's key-loading interface, provisioning workflow, and recovery design before claiming keys cannot be extracted. A TPM cannot by itself make a compromised authorized application safe.

Credential operations checklist

Operation	**Required decision or artifact**
Provision	Identity owner, enclave path, subject, permissions, authority certificates.
Deploy	Required runtime files only, resolved symlinks, actual service account and file access.
Renew	Certificate/permission validity, clock source, rollout and rollback sequence.
Compromise	Isolate participant, replace credentials/policy, assess exposed keys and affected robots.
Recover	Restore trusted software and verify denied/authorized behavior before reconnecting.
Audit	Retained hashes, versions, issuance/change records and sanitized logs.

Revocation behavior is implementation dependent. Test how a peer handles changed permissions and revoked or expired material, including already-established sessions and restarts. Document what must happen to remove access; do not promise immediate revocation from a paper policy alone.

Exercise and explain-back

Draw the node → context → participant → enclave → process/account mapping for two components. List the files each account can read and the union of operations each identity can perform. Then add a third-party plugin to one process and redraw the trust boundary.

Artifact: mapping diagram, deployment file list, key-access review, and renewal/recovery procedure.

Explain back: Why is `key.pem` secret even after removing `private/`? Why does an absolute enclave path not select a ROS namespace? When would two processes still share an unsafe credential boundary?

Misconceptions: SROS 2 enclaves are not hardware enclaves; public certificate files are not signing authority keys; separate paths do not create separate process memory.

Limit: this is a reviewed design exercise, not proof of host isolation or physical hardware protection. Next, connect the mapping to the threat records in the threat-model chapter.

Sources

- ROS security setup and keystore root.
- ROS command-line enclave arguments.
- Node-to-participant design.
- SROS 2 project.

ROS 2 Robotic Systems Threat Model

Original authors: Thomas Moulard, Juan Hortala Xabi Perez Gorka Olalde Borja Erice Odei Olalde David Mayoral

2026 revision, security review, and additional material: Juan Carlos Nevarez · Gh0stlyKn1ght

Originally written: 2019-03 · Revised: 2026-10-20

Original article.

Goal and prerequisites

Build a threat model that connects an attacker action to an asset, a trust boundary, a proposed control, and evidence needed to assess the remaining risk. Read the security, policy, and enclaves chapters first. You should be able to describe the robot's mission and its process/network architecture.

Practical question: An attacker steals a legitimate participant's key. Which controls still protect the robot, and what evidence would show that they work? Encryption of traffic is only one part of the answer.

Scope and edition status

The current worked model is a **non-actuating talker/listener teaching system** using the selected Jazzy/DDS configuration described in the first chapter. It is a bounded example, not a proxy for a production robot. Its policy can be checked offline; live communication and safety behavior require separate evidence.

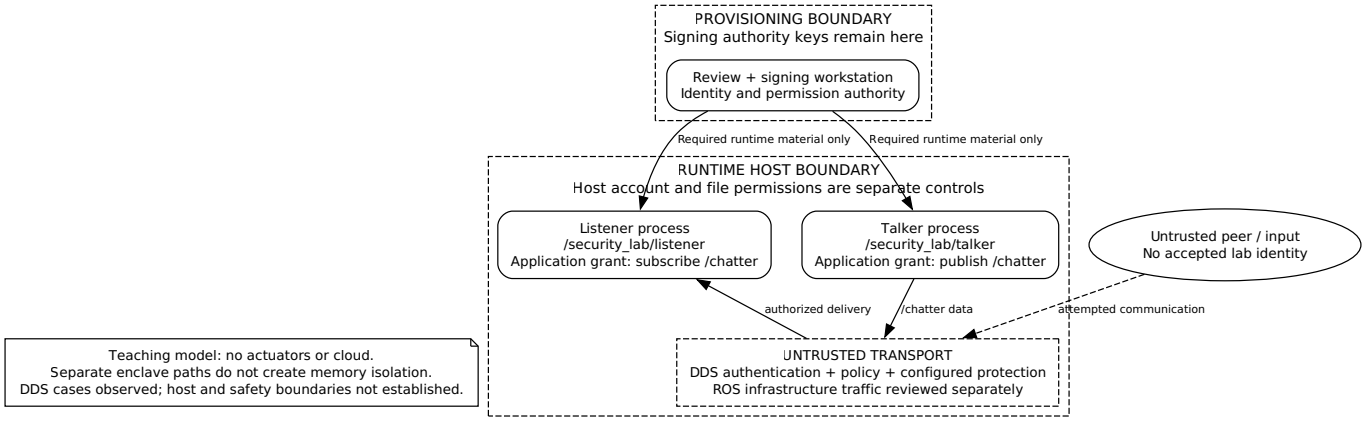
The original TurtleBot 3 and MARA models used Crystal-era configurations and include third-party historical assessment results. Those results and the original threat catalog are preserved in the historical source and historical reader. They are not evidence that a current platform is exploitable, fixed, or safe. This revision replaces their role as the default implementation baseline while retaining their value as case studies.

Model the system before scoring it

Record the mission, operator, deployment environment, approved maintenance paths, version inventory, and exclusions. Separate known observations from assumptions. For a robot, include actuators, sensors, embedded controllers, local administration, firmware updates, cloud services, and external data providers where they actually exist.

An **asset** is something to protect: control authority, availability, credentials, private data, software integrity, or human safety. An **entry point** accepts influence: a DDS topic, a remote shell, a dashboard, USB, a sensor, or an update pipeline. A **trust boundary** is where assumptions about identity, privilege, or enforcement change. A dashed box on a diagram is only meaningful when an actual control enforces it.

Worked architecture



The provisioning workstation signs identity/policy material. The runtime system has separate talker and listener enclave paths and processes. The application flow is `/chatter`; ROS infrastructure traffic must be accounted for separately. The network is treated as untrusted even when the exercise runs on one host. There are no actuators and no cloud dependencies in this model.

Boundary	**Assumption**	**Control and evidence required**
Provisioner → runtime	Authority private keys stay on the provisioner.	Deployment file inventory and service-account read checks.
Process → DDS network	Peers are authenticated and operations are authorized.	Reviewed governance/permissions plus positive and negative runtime cases.
Talker ↔ listener host resources	Processes cannot freely read each other's secrets.	OS identity/access design; this is not provided by enclave names.
Incoming message → application	Authenticated data can still be invalid.	Application validation and bounded processing; the toy listener does not establish robot safety.

Use STRIDE consistently

STRIDE is **Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege**. The original prose incorrectly used “Integrity” for the “I”; the corrected category is Information Disclosure. Integrity remains an important security property, but it is not that category name. See Microsoft's STRIDE definitions.

Apply the categories to the actual data flows and stores rather than treating the acronym as a complete list of robot hazards. A threat can span multiple categories. For example, stealing a key can lead to information disclosure and spoofing, while an authorized process flooding a control path can cause denial of service.

Risk and evidence fields

Each current threat record includes an ID, attacker prerequisite, affected asset/boundary, scenario, control, verification action, owner role, treatment, evidence status, and residual-risk note. An owner role in this teaching model is a responsibility to assign, not a named person who has accepted it. Do not invent exploitation probabilities or percentages.

Use likelihood rationale and mission-specific impact for deployment decisions. The original DREAD totals and MARA RVSS results remain historical annotations in the archive. For a concrete vulnerability, CVSS can supplement severity discussion; it does not replace operational risk or a robot safety assessment. FIRST's CVSS 4.0 specification includes safety-related metrics, but a number alone is not a safety argument.

Use these evidence states:

- **Proposed:** a design or test case exists, but no implementation evidence is supplied.
- **Offline-checked:** a file or transformation passed the named local check.
- **Runtime-observed:** a recorded software run produced the stated outcome on exact versions.
- **Bench-observed:** an isolated device run produced the stated outcome.
- **Robot-observed:** the stated behavior was observed on the identified physical system.
- **Historical:** a past assessment or design claim is retained with its original scope/date.

“Mitigated” requires an explanation of which attack path is blocked under which assumptions and what risk remains. Do not convert a proposed mitigation into a completed result by changing the wording.

Current teaching-system threat register

The canonical records are in threat-register.json. Their full text is included in the updated handbook PDF. The summary below is an index, not a replacement for the records.

ID	Threat and prerequisite	Control / validation focus
TM-001	Untrusted participant attempts to inject `/chatter` data.	Authentication, governance admission and restrictive grants; untrusted-keystore case.
TM-002	Authenticated listener attempts application publishing.	Direction-specific grants and default denial; inspect generated XML and test forbidden publish.
TM-003	Missing or unreadable enclave material causes unsecured startup.	Deliberate enable/enforce settings; startup failure case.
TM-004	Local attacker obtains an enclave private key.	OS/file boundaries, key replacement and isolation; review actual service-account access.
TM-005	Authorized participant supplies excessive or invalid input.	Application bounds, resource controls and recovery; bounded load tests on the selected host.
TM-006		

ID	Threat and prerequisite	Control / validation focus
	Compromised update/provisioning path installs trusted malicious code or policy.	Reviewed signed inputs, artifact inventory, separate signing roles and recovery.
TM-007	Logs/inspection reveal sensitive data or broaden access.	Separate inspection identity, retention/access policy and sanitized evidence.
TM-008	Clock/expiry or credential change disrupts communication.	Validity checks, trusted time assumptions, renewal and restart/ recovery tests.

The offline checker supplies evidence only for relevant XML structure and grant-scope assertions. The recorded runtime run additionally observed application-write denial and missing-enclave failure, chiefly TM-002 and TM-003. See the runtime observations. It does not execute the network attacks, establish confidentiality, or prove any robot is safe. Every unexecuted runtime test remains **not run**.

Validate claims with paired cases

Start with authorized communication on the same host/configuration. Then vary one property at a time: enclave presence, trusted authority, topic permission, or application input. Record exit code, sanitized logs, actual message delivery, elapsed observation window, and exact versions. Use logs and a positive control to distinguish rejection from a broken network or a QoS/type mismatch.

For packet inspection, define which interface and flows were captured, which protection settings were enabled, and what was visible. Not seeing plaintext in one capture is not proof that every channel or metadata field is confidential. Avoid capturing unrelated users' traffic or retaining credentials in shared evidence.

Tests that can affect motion belong on a separately bounded hardware test plan with an independent stop mechanism and recovery procedure. The provided non-actuating lab does not issue movement commands.

Extend the model to an actual robot

1. Name the robot, firmware, OS, ROS distribution, RMW, middleware/security libraries, packages, topology and mission.
2. Inventory operators, remote support, provisioning, cloud connections, sensors, actuators, buses and update sources.
3. Draw data flows, stores and actual enforcement boundaries; identify safety-critical paths separately.
4. Start from the historical threat catalog and current records, then remove irrelevant assumptions and add deployment-specific scenarios.
5. Assign owner roles and treatment: reduce, avoid, transfer or explicitly accept with an accountable decision and review date.
6. Pair each control with authorized and rejected cases, recovery observations, evidence location and a residual-risk statement.
7. Revisit the model after architecture, privilege, firmware, policy, or exposure changes.

Host, network and supply chain

Use supported software, minimal service privileges, credential separation, reviewed network paths and a defined update process. Record build inputs and dependencies; an SBOM is an inventory aid, not proof that software is trustworthy. Verify update signatures before installation, protect signing keys, and design recovery and rollback deliberately. A validly signed malicious or vulnerable update remains possible when the signing/provisioning authority is compromised.

ROS bridges, dashboards, VPN gateways, microcontroller agents, serial links and CAN buses each create their own boundary. Identify where authentication/encryption terminate and whether a bridge can republish into a more trusted domain. Do not assume DDS protection extends across a non-DDS link.

Safety, privacy and optional AI

A robot needs appropriate command validation, stale-input handling, watchdog/degraded-mode behavior, resource limits and independent stop controls. Security and functional safety share some failure consequences but require their own evidence. The absence of a reported vulnerability is not proof that movement is safe.

Record what sensor/log data is collected, who can access it, retention and deletion, and what is exported off the robot. If the system uses learned perception or language-model tools, model untrusted inputs, sensor deception, tool authority and output validation as additional flows. Do not add AI-specific controls to a system that has no such component merely to make the model look modern.

Exercise, artifact and next step

Use the supplied threat-record format to model one maintenance operation, such as a policy renewal or software update. Draw the boundary, write an attacker prerequisite, propose a control, and state exactly what observation would support the claim. Leave its status proposed until that evidence exists.

Artifact: mission/scope sheet, diagram, threat register, version manifest and evidence log.

Explain back: Which threats remain after enabling encryption? What changes when two components share a process? Why is a historical MARA finding not automatically a finding on your robot?

Misconceptions: a scanner's clean result is not a complete threat model; a risk score is not a test result; a planned control is not implemented evidence; authentication does not establish safe command semantics.

Limit: this edition includes documentation review, offline validation, and bounded software runtime observations. Six ROS cases passed; the separate-authority case is inconclusive. No physical robot validation was performed. Next, repeat the bounded lab on the target deployment and attach its observations before extending any claim to that system.

Sources and historical attribution

- Original ROS 2 threat model at the reviewed revision.
- Microsoft STRIDE.
- FIRST CVSS 4.0.
- OWASP Threat Dragon for optional collaborative/local model editing.
- Current security tooling choices.

Original threat-model authors remain credited above. The MARA assessment described in the archive was performed by Alias Robotics for the named historical system; this fork's revision is not a new assessment of that platform.

Full teaching-system threat records

Owner roles are responsibilities to assign. Offline-checked status covers named local assertions only. The runtime observations record six passed ROS cases and one inconclusive separate-authority case. Other controls remain proposed.

TM-001 · Untrusted participant injects application messages

Stride
Spoofing / Tampering

Attacker prerequisite
Attacker can reach the selected DDS transport but does not hold a trusted lab identity.

Asset and boundary
Application data integrity; process-to-network boundary.

Scenario
A peer attempts to publish /chatter without an identity accepted by the intended trust domain.

Control
Require authenticated admission, reviewed governance and restrictive signed grants.

Verification action
Establish authorized delivery, then repeat with an independently created authority and record handshake/rejection evidence.

Owner role
Middleware / PKI maintainer

Evidence status
proposed

Evidence
Separate-authority delivery was not observed; explicit authentication rejection remains inconclusive. See runtime-observations.md.

Residual risk
A compromised trusted identity can still publish within its grants.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-002 · Listener identity gains application publishing authority

Stride
Elevation of Privilege / Tampering

Attacker prerequisite
An authenticated process holds the listener identity; policy authoring or deployment could grant unnecessary writes.

Asset and boundary
Application publishing authority; policy-to-transport boundary.

Scenario
The listener is accidentally granted publication on /chatter or a wildcard that includes it.

Control
Generate an explicit listener read grant and default denial; inspect the deployed signed policy.

Verification action
Check exact generated publish/subscribe sets; attempt a forbidden application publication on the selected ROS host.

Owner role
Policy reviewer

Evidence status
runtime-observed

Evidence
reports/security-handbook/runtime-evidence/results.json and named logs. See runtime-observations.md for exact scope and remaining limits.

Residual risk
The local check proves intended generated scope; it does not prove the runtime loaded or enforced that scope.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-003 · Missing credentials lead to unsecured startup

Stride
Spoofing / Information Disclosure

Attacker prerequisite
An operator or service starts a process with absent/unreadable enclave artifacts or an incorrect inherited configuration.

Asset and boundary
Confidentiality and trusted communication; launch-to-middleware boundary.

Scenario
A missing keystore or enclave triggers permissive fallback instead of an enforced startup failure.

Control
Set ROS_SECURITY_ENABLE=true and ROS_SECURITY_STRATEGY=Enforce in the actual service environment; track enclave overrides.

Verification action
Run the missing-enclave case and record exit status and initialization errors.

Owner role
Deployment maintainer

Evidence status
runtime-observed

Evidence
reports/security-handbook/runtime-evidence/results.json and named logs. See runtime-observations.md for exact scope and remaining limits.

Residual risk
An external service manager can still supply the wrong environment; inspect its effective settings.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-004 · Local code obtains a runtime private key

Stride
Information Disclosure / Spoofing

Attacker prerequisite
Attacker gains readable filesystem/memory access through an overprivileged account, component or mount.

Asset and boundary
Participant identity; OS/process/credential boundary.

Scenario
Readable key.pem is copied and used to impersonate the participant elsewhere.

Control
Separate runtime accounts/trust scopes, restrict file access and mounts, keep authority keys off runtime hosts, define compromise recovery.

Verification action
Check read access as each actual account and exercise isolated credential replacement/recovery.

Owner role
Host / PKI maintainer

Evidence status
proposed

Evidence
Runtime key modes were 0600 in the namespace; no host-isolation or key-theft simulation was performed.

Residual risk
Code in the same trusted process can use its identity; encryption of traffic does not protect a stolen key.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-005 · Authorized input exhausts resources or violates application assumptions

Stride
Denial of Service / Tampering

Attacker prerequisite
A participant has an allowed input path or an attacker reaches a resource-constrained interface.

Asset and boundary
Availability and semantic correctness; message-to-application boundary.

Scenario
A trusted sender delivers excessive, stale or semantically invalid inputs.

Control
Bound queues and processing, validate data and timing, define degraded operation and recovery.

Verification action
Use a bounded non-actuating load/input test with resource measurements and recovery observations.

Owner role
Application / operations maintainer

Evidence status
proposed

Evidence
No runtime load or application validation tests supplied.

Residual risk
The toy demo does not implement a robot safety policy or independent stop control.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-006 · Provisioning or update authority introduces trusted malicious artifacts

Stride
Tampering / Elevation of Privilege

Attacker prerequisite
Attacker controls an approved build input, signing role, update source or policy approval path.

Asset and boundary
Software and policy integrity; supply-chain/provisioning boundary.

Scenario
A malicious binary or overly broad policy is installed with apparently valid provenance/signatures.

Control
Review inputs, separate update/identity/permission signing roles, record artifact/dependency inventory, constrain signing and provide recovery.

Verification action
Review the trust chain and exercise rejection of an unauthorized artifact plus a recovery path in an isolated environment.

Owner role
Build / release / PKI maintainer

Evidence status
proposed

Evidence
Vendored XML inputs are hash-checked locally; no production update pipeline was assessed.

Residual risk
A valid signature proves a relationship to a signing key, not that the signed content is harmless.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-007 · Inspection and logs expose data or broaden operational access

Stride
Information Disclosure / Repudiation

Attacker prerequisite
An operator, compromised logger or external support tool can access logs or inspection credentials.

Asset and boundary
Sensitive messages, credentials and audit evidence; administration/data-retention boundary.

Scenario
A copied production identity or unrestricted evidence bundle discloses data or permits extra actions.

Control
Use a separate reviewed inspection identity, scoped log access, retention rules and sanitized shared evidence.

Verification action
Review actual inspection grants and evidence contents; test that an inspection principal cannot issue control operations.

Owner role
Operations / data owner

Evidence status
proposed

Evidence
No live inspection identity or retention mechanism was tested.

Residual risk
Logs may be incomplete or modified on a compromised host; retain provenance and assess reliability.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

TM-008 · Clock, expiry or credential replacement disrupts communication

Stride
Denial of Service / Spoofing

Attacker prerequisite
Clock state changes, credentials expire, or a renewal/revocation rollout is incomplete.

Asset and boundary
Availability and identity lifecycle; time/provisioning/runtime boundary.

Scenario
Peers reject otherwise intended communication or retain access longer than the recovery plan assumes.

Control
Review validity intervals, time assumptions, renewal overlap, replacement and peer/session restart behavior.

Verification action
Exercise expiry/renewal and removal of an identity on the chosen implementation, including existing sessions and recovery.

Owner role
PKI / operations maintainer

Evidence status
proposed

Evidence
Offline example validity is illustrative; no time or revocation runtime test was performed.

Residual risk
Do not assume instantaneous revocation or an online revocation service across vendors.

Treatment
reduce (proposed until demonstrated)

Deployment risk
Unassessed: assign likelihood rationale and mission-specific impact on the target deployment.

Review date
2026-10-20

Libraries and technology choices

Juan Carlos Nevarez · Gh0stlyKn1ght · Reviewed October 20, 2026.

Use SROS 2 for DDS identity and permissions, a bounded XML checker for the example policy, and an editable threat register and architecture diagram. The choices below distinguish tools used in the lab from deployment options that need a target-system trial.

Tools used in the lab and threat model

Technology	Purpose and reason	Limit
SROS 2 policy 0.2.0	ROS-oriented privileges and a path to signed DDS permission artifacts. The offline example uses a pinned upstream schema and transform.	A valid policy and signed file still need actual middleware enforcement tests.
Python + lxml	Schema validation, the upstream XSLT, exact grant assertions and input-boundary checks. Schema resolution is local; DTDs and XInclude are rejected for this example.	The checker accepts this defined format, not arbitrary external XML/XSLT.
Graphviz DOT → SVG/PDF	Editable architecture source with explicit processes, assets and trust boundaries.	Diagram boundaries are design claims until their controls are verified.
JSON threat register and result template	Stable IDs, prerequisites, owners, evidence states and residual-risk notes, rendered into printable records.	No automatic risk score substitutes for a deployment-specific judgment.

Primary documentation: SROS 2, lxml resolution and XSLT access control, and Graphviz SVG.

Middleware and platform decisions

For the included exercise: use Jazzy on Ubuntu 24.04 with `rmw_fastrtps_cpp`. This keeps the commands and policy aligned with a reviewed official tutorial and the pinned Jazzy transformation. Capture installed package versions; do not infer package versions from the ROS distribution name. The isolated run recorded six passed ROS cases and one inconclusive separate-authority case; consult the runtime observations before relying on a particular claim.

For a new long-lived robot: evaluate Lyrical on Ubuntu 26.04 when its hardware/application dependencies are supported. Lyrical's official support horizon is May 2031; Jazzy remains listed through May 2029. The decision depends on drivers, vendor support, package availability, and a measured migration run. See the official release table and Lyrical platform matrix.

Fast DDS is a suitable starting point for this DDS-focused lab. **Cyclone DDS** is a credible alternate implementation for a separately tested deployment; both document security plugins and signed governance/permissions. Neither is declared “more secure” here, and no performance benchmark was run. Choose based on supported packaging, required features, operational evidence and vendor support. The Jazzy tutorial does not support secure cross-vendor communication. Sources: Fast DDS security, Cyclone DDS security, and Jazzy tutorial.

Zenoh is an option when its routing/topology model fits the application. The current `zenoh_security_tools` can generate session configs using SROS 2 policy and keystore material. Its router/session settings and bridge boundaries need their own review; switching `RMW_IMPLEMENTATION` is not a complete migration of DDS security. Source: official Zenoh security tools. This option is documented, not installed or validated in this package.

Useful next-stage tools

- **OWASP Threat Dragon desktop:** useful when several people want a visual data-flow editor with threat/remediation records and local files. Keep the JSON register as a reviewable artifact and check export compatibility with old diagrams. It is optional; the authored DOT/SVG figure already works offline. Threat Dragon documentation.
- **Syft + Grype:** useful for dependency inventory and vulnerability triage of a selected deployment image/filesystem. Record tool versions, artifact digest, database age and exceptions. An SBOM is not evidence that a vulnerability is exploitable; a clean scan does not prove safety. These tools were researched, not run. Syft, Grype.
- **Cosign:** useful when an actual release pipeline needs artifact-signature verification. Define the accepted signing identity/issuer or trusted key and verify the intended artifact digest. Keep this release-signing trust separate from DDS identities. No signing integration was configured. Cosign verification.

The most useful next investment is stronger instrumentation for the inconclusive separate-authority case, followed by a small current-robot case study. Introducing more tooling before those results would not strengthen the handbook's evidence.

Bounded SROS 2 teaching lab

Juan Carlos Nevarez · Gh0stlyKn1ght · October 20, 2026.

Goal: distinguish trusted identity, permitted data flow, and enforced startup using a talker/listener system with no actuators. The exercise targets Ubuntu 24.04 + ROS 2 Jazzy + `rmw_fastrtps_cpp`. It was exercised in isolated Ubuntu userspace; see the recorded lab results for six passed ROS cases and one inconclusive case. Lyrical, Cyclone DDS, and Zenoh require separately recorded compatibility runs.

Prerequisites and boundaries

Use an isolated lab host with the Jazzy environment sourced and the distribution's `sros2`, `demo_nodes_cpp`, `rmw_fastrtps_cpp`, and Python `lxml` packages installed. Follow the official ROS installation instructions for your platform. Confirm that the chosen DDS package includes security support. Do not attach a moving robot. The exercise uses domain 42 and localhost discovery; neither setting is an authorization control.

The setup script creates signing keys and node private keys in a **new** directory with restrictive creation permissions. It refuses to overwrite an existing path. Keep the entire provisioning directory private, outside version control. The repository contains policy and unsigned example permissions only; it never contains a real keystore.

The policy grants `/chatter` publishing only to the talker, and subscription only to the listener. Both identities have explicit logging/parameter/time allowances and replies for their own parameter/type-description services. The transform adds the DDS ROS discovery topic. These allowances are part of the example's attack surface; the listener is not a principal that can perform no publishing at all.

1. Run the offline check

From the repository root:

```
python3 examples/security-lab/check_policy.py
python3 -m unittest discover -s examples/security-lab -p 'test_*.py' -v
```

The checker validates the policy and generated DDS XML against pinned schemas. It verifies exact topic direction sets, certificate subjects, domain 42, default denial, and no wildcard grants. It rejects DTD/entity declarations and XInclude, disables network resolution, and denies XSLT I/O. The vendored files have a source manifest and hashes. This protects the local check's defined input boundary; it is not a generic sandbox for arbitrary XML/XSLT.

`generated/permissions.xml` is **unsigned** and has an illustrative October 2026–October 2027 interval. It is evidence of transformation, not deployable credentials. Runtime SROS 2 derives permission validity from the actual certificate. Review that validity and use a deployment-specific renewal policy.

2. Provision on the ROS host

Choose a new absolute directory and run:

```
bash examples/security-lab/setup.sh "$HOME/ros2-security-lab"
```

The script runs the offline check before creating a keystore, creates separate talker/listener enclaves, and applies `policy.xml` using `create_permission`. It records package versions in the private lab directory. Check the capture result: package names differ, so resolve missing entries before treating the version inventory as complete. Review the generated `governance.xml`, `permissions.xml`, signed files, certificate subjects, domains and validity before proceeding. Capture their hashes, not their private-key contents. A new example keystore is not a production signing service.

3. Establish the authorized control case

In **each** terminal, source ROS Jazzy, then:

```
source examples/security-lab/environment.sh
export ROS_SECURITY_KEYSTORE="$HOME/ros2-security-lab/keystore"
```

Run the talker in terminal A:

```
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/talker
```

Run the listener in terminal B:

```
ros2 run demo_nodes_cpp listener --ros-args --enclave /security_lab/listener
```

Observe several messages, record the time window and logs, and stop both with Ctrl-C. If this control case fails, resolve credentials, governance, clock, version, network, type and QoS problems before interpreting a negative case.

4. Test denied behavior

Keep the same environment and vary one property at a time.

Missing enclave: use an enclave that was never created:

```
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/missing
```

Expected: the secured participant fails to initialize. Record the actual error and exit status. A fallback to unsecured startup fails this case.

Forbidden remap: retain the talker identity but change the application topic:

```
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/talker \
  --remap chatter:=not_chatter
```

Expected: the unauthorized endpoint is rejected and no forbidden messages are delivered. The process-level outcome can depend on implementation and node error handling. Record what occurs rather than copying an expected log message.

Wrong direction: run the talker executable under the listener enclave:

```
ros2 run demo_nodes_cpp talker --ros-args --enclave /security_lab/listener
```

Expected: this identity is denied publication on `/chatter`. The executable's default node name also changes its parameter-service names relative to the listener policy, so this is a whole-profile rejection case. The exact generated grant check separately isolates the application-direction assertion.

Untrusted authority: create a separate fresh lab directory with the setup script. Start the original talker using the first keystore; start the listener using the second keystore, with the same listener enclave path. The independently generated authorities should prevent trusted communication. Record the handshake or access evidence and compare it with the authorized control. Successful startup alone does not establish peer acceptance.

5. Record results honestly

Case	Required observation	Initial status
Offline policy/grant check	Named local assertions pass.	Run by the package author; see generated check record.
Authorized pair	Listener receives known talker messages.	Seven messages observed; pass.
Missing enclave	Enforced initialization failure.	Observed; exit 250, pass.
Forbidden remap	Forbidden endpoint/delivery denied.	Writer creation rejected; pass.
Wrong identity profile	Unauthorized application publication denied.	Profile rejected; focused write probe also rejected; pass.
Separate authority	Untrusted peer cannot exchange the protected application data.	Zero messages, no explicit rejection evidence; inconclusive.

These are recorded author-run observations, not guaranteed outcomes on your host. Repeat the checks and retain your own results. The runtime runner can automate these bounded cases.

Copy `results-template.json` into the private lab directory, record each actual outcome, and link sanitized logs plus the version manifest. “No messages” without a working authorized control and rejection evidence is inconclusive. Do not change a result to pass because XML was generated successfully.

Explain back: What is authenticated? Which principal receives permissions? Why does default denial not stop an authorized process abusing an allowed topic? Which host file access would expose the identity?

Artifact: diagram, threat record, reviewed policy/governance, artifact hashes, version manifest, observed cases and limitations. Next, connect one deployment-specific robot control to the threat register and a separately bounded hardware test plan.

Sources and notices

SROS 2, the official Jazzy security setup, and access controls inform the procedure. Vendored SROS 2 schemas/transform retain Apache 2.0 notices in `vendor/sros2/LICENSE`; W3C schema attribution and its source link are recorded in the vendor manifest and notice. The new scripts and examples use the repository's Apache 2.0 license. Handbook prose retains the design-document attribution context described in the updated PDF.

Optional runtime automation

`run_runtime.py` executes the bounded software cases against two independently provisioned keystores. See the recorded runtime observations for its exact command, result interpretation and the remaining inconclusive case. It returns nonzero for failed or inconclusive cases. `probe_publish.py` isolates application writer creation after the listener node has initialized successfully. `authentication_probe.cpp` is a separate Fast DDS 2.14 callback diagnostic; its trusted acceptance and silent untrusted case are recorded separately from the ROS cases.

Recorded lab results

Juan Carlos Nevarez · Gh0stlyKn1ght · October 20, 2026.

What was actually run

The non-actuating lab ran in Ubuntu 24.04 Noble userspace on x86_64 with official ROS Jazzy packages. Tests used a private network namespace with loopback only. This was a software communication test; host isolation and physical robot behavior were outside its scope.

The installed baseline included **Fast DDS 2.14.7**, **rmw_fastrtps_cpp 8.4.4**, and **SROS 2 0.13.6**. Exact Debian package versions, the host kernel, selected settings, public runtime-artifact hashes, and sanitized logs are retained in `runtime-evidence/`. Authority keys and participant private keys remain outside the repository.

Observed ROS cases

Case	Observation	Result
Authorized pair	Seven messages published and received during the observation window; both participants stayed running.	Pass.
Missing enclave	Initialization failed without an artificial test shutdown; exit 250.	Pass.
Forbidden topic remap	Publisher creation failed; no application messages published; exit 250.	Pass.
Wrong identity profile	The talker executable under the listener identity failed to initialize its allowed profile; exit 250.	Pass, for whole-profile rejection.
Focused listener publishing probe	The listener node was constructed successfully, then `/chatter` writer creation was explicitly denied by Fast DDS; probe exit 2.	Pass, for application write denial.
Independently generated authority	Talker published seven messages; listener received zero. Both stayed running, but no explicit authentication rejection appeared in default logs.	**Inconclusive** for the claimed rejection mechanism.
Authorized control repeated afterward	Seven messages published and received after the negative cases.	Pass.

Six of seven ROS cases passed. The complete suite is not reported as passing. The separate-authority observation meets the no-delivery expectation but does not by itself establish why delivery stopped.

A supplementary native Fast DDS probe recorded `AUTHENTICATION_ACCEPTED` for the trusted peer. The same probe with the independently created authority recorded neither acceptance nor rejection in its eight-second window. That result is retained, not converted into a rejection claim. Further discovery/handshake instrumentation is needed before closing this evidence gap.

The actual permissions XML generated by the installed SROS 2 package also passed the exact two-grant checks used by the offline example; see `runtime-permission-check.json`.

The two runtime enclave keys had mode `0600` in this environment. That checks group/other mode bits; it does not establish that privileged code, same-process code, mounts, or a different deployment account cannot access them.

Reproduce the software tests

On a prepared Jazzy host, provision two new lab directories using the documented setup script. Then run the runner with their separate keystores and a new output directory:

```
python3 examples/security-lab/run_runtime.py \
  --keystore /absolute/first/keystore \
  --untrusted-keystore /absolute/second/keystore \
  --output /absolute/new-results-directory
```

Run only on a non-actuating lab system. The runner uses domain 42 and localhost discovery, scopes shutdown to the processes it starts, refuses to reuse an output directory, and returns a nonzero status if any case fails or is inconclusive. Review the logs and version manifest before comparing results across machines.

The optional `authentication_probe.cpp` targets the Fast DDS 2.14 API and requires that version's headers/libraries and a C++ compiler. Its native callback observations are separate from the ROS application delivery cases; it does not replace them.

Limits of these observations

The separate-authority case remains inconclusive. Further tests are needed for revocation, expiry, credential renewal and recovery, hardware-backed keys, privileged-host isolation, bounded overload behavior, and packet-level confidentiality. No physical robot or actuator was tested. The run does not establish a vulnerability assessment or compliance conclusion.

Complete example policy

Source: examples/security-lab/policy.xml. Offline-checked scope for the selected example; not signed runtime credentials.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Juan Carlos Nevarez / Gh0stlyKn1ght, 2026. Non-actuating teaching policy. -->
<policy version="0.2.0">
  <enclaves>
    <enclave path="/security_lab/talker">
      <profiles>
        <profile ns="/" node="talker">
          <topics publish="ALLOW">
            <topic>/chatter</topic>
            <topic>/rosout</topic>
            <topic>/parameter_events</topic>
          </topics>
          <topics subscribe="ALLOW">
            <topic>/parameter_events</topic>
            <topic>/clock</topic>
          </topics>
          <services reply="ALLOW">
            <service>~/describe_parameters</service>
            <service>~/get_parameter_types</service>
            <service>~/get_parameters</service>
            <service>~/list_parameters</service>
            <service>~/set_parameters</service>
            <service>~/set_parameters_atomically</service>
            <service>~/get_type_description</service>
          </services>
        </profile>
      </profiles>
    </enclave>
    <enclave path="/security_lab/listener">
      <profiles>
        <profile ns="/" node="listener">
          <topics publish="ALLOW">
            <topic>/rosout</topic>
            <topic>/parameter_events</topic>
          </topics>
          <topics subscribe="ALLOW">
            <topic>/chatter</topic>
            <topic>/parameter_events</topic>
            <topic>/clock</topic>
          </topics>
          <services reply="ALLOW">
            <service>~/describe_parameters</service>
            <service>~/get_parameter_types</service>
            <service>~/get_parameters</service>
            <service>~/list_parameters</service>
            <service>~/set_parameters</service>
            <service>~/set_parameters_atomically</service>
            <service>~/get_type_description</service>
          </services>
        </profile>
      </profiles>
    </enclave>
  </enclaves>
</policy>
```

Credits and license notices

2026 revision and new material: Juan Carlos Nevarez · Gh0stlyKn1ght. Original design-document attribution: Open Source Robotics Foundation, Inc. Original article authors remain identified in the revised chapters.

The original site specifies Creative Commons Attribution 3.0 for design documents except where otherwise noted. This edition preserves that attribution context and identifies changes. CC BY 3.0.

Original source snapshots are stored in articles/security_history. The historical reader remains in reports/security-review. The project also supplies an Apache 2.0 code license; its full text follows. New example scripts and policy are under the repository code license.

Vendored XML tooling

The unchanged SROS 2 schemas and permission transform come from ros2/sros2 commit `80a08ffa1bccd2de602c2acd120b209ad5f8ce83` (Jazzy branch, retrieved October 20, 2026). Copyright and license context: SROS 2 / Open Source Robotics Foundation and its contributors, Apache License 2.0; the upstream license is reproduced in `sros2/LICENSE`. Exact paths, URLs and SHA-256 hashes are recorded in `sources.json`. These files support the bounded offline example; their inclusion does not certify interoperability or current middleware behavior.

`xml.xsd` is the W3C XML namespace schema from https://www.w3.org/2001/03/xml.xsd. The schema is unchanged; its source URL and hash are retained. License: https://www.w3.org/Consortium/Legal/2002/copyright-software-20021231 .

Repository code license


```
                                 Apache License
                           Version 2.0, January 2004
                        http://www.apache.org/licenses/

   TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

   1. Definitions.

      "License" shall mean the terms and conditions for use, reproduction,
      and distribution as defined by Sections 1 through 9 of this document.

      "Licensor" shall mean the copyright owner or entity authorized by
      the copyright owner that is granting the License.

      "Legal Entity" shall mean the union of the acting entity and all
      other entities that control, are controlled by, or are under common
      control with that entity. For the purposes of this definition,
      "control" means (i) the power, direct or indirect, to cause the
      direction or management of such entity, whether by contract or
      otherwise, or (ii) ownership of fifty percent (50%) or more of the
      outstanding shares, or (iii) beneficial ownership of such entity.

      "You" (or "Your") shall mean an individual or Legal Entity
      exercising permissions granted by this License.

      "Source" form shall mean the preferred form for making modifications,
      including but not limited to software source code, documentation
      source, and configuration files.
```

```
      "Object" form shall mean any form resulting from mechanical
      transformation or translation of a Source form, including but
      not limited to compiled object code, generated documentation,
      and conversions to other media types.

      "Work" shall mean the work of authorship, whether in Source or
      Object form, made available under the License, as indicated by a
      copyright notice that is included in or attached to the work
      (an example is provided in the Appendix below).

      "Derivative Works" shall mean any work, whether in Source or Object
      form, that is based on (or derived from) the Work and for which the
      editorial revisions, annotations, elaborations, or other modifications
      represent, as a whole, an original work of authorship. For the purposes
      of this License, Derivative Works shall not include works that remain
      separable from, or merely link (or bind by name) to the interfaces of,
      the Work and Derivative Works thereof.

      "Contribution" shall mean any work of authorship, including
      the original version of the Work and any modifications or additions
      to that Work or Derivative Works thereof, that is intentionally
      submitted to Licensor for inclusion in the Work by the copyright owner
      or by an individual or Legal Entity authorized to submit on behalf of
      the copyright owner. For the purposes of this definition, "submitted"
      means any form of electronic, verbal, or written communication sent
      to the Licensor or its representatives, including but not limited to
      communication on electronic mailing lists, source code control systems,
      and issue tracking systems that are managed by, or on behalf of, the
      Licensor for the purpose of discussing and improving the Work, but
      excluding communication that is conspicuously marked or otherwise
      designated in writing by the copyright owner as "Not a Contribution."

      "Contributor" shall mean Licensor and any individual or Legal Entity
      on behalf of whom a Contribution has been received by Licensor and
      subsequently incorporated within the Work.

   2. Grant of Copyright License. Subject to the terms and conditions of
      this License, each Contributor hereby grants to You a perpetual,
      worldwide, non-exclusive, no-charge, royalty-free, irrevocable
      copyright license to reproduce, prepare Derivative Works of,
      publicly display, publicly perform, sublicense, and distribute the
      Work and such Derivative Works in Source or Object form.

   3. Grant of Patent License. Subject to the terms and conditions of
      this License, each Contributor hereby grants to You a perpetual,
      worldwide, non-exclusive, no-charge, royalty-free, irrevocable
      (except as stated in this section) patent license to make, have made,
      use, offer to sell, sell, import, and otherwise transfer the Work,
      where such license applies only to those patent claims licensable
      by such Contributor that are necessarily infringed by their
      Contribution(s) alone or by combination of their Contribution(s)
      with the Work to which such Contribution(s) was submitted. If You
      institute patent litigation against any entity (including a
      cross-claim or counterclaim in a lawsuit) alleging that the Work
      or a Contribution incorporated within the Work constitutes direct
      or contributory patent infringement, then any patent licenses
      granted to You under this License for that Work shall terminate
      as of the date such litigation is filed.

   4. Redistribution. You may reproduce and distribute copies of the
      Work or Derivative Works thereof in any medium, with or without
      modifications, and in Source or Object form, provided that You
      meet the following conditions:

      (a) You must give any other recipients of the Work or
          Derivative Works a copy of this License; and

      (b) You must cause any modified files to carry prominent notices
          stating that You changed the files; and

      (c) You must retain, in the Source form of any Derivative Works
          that You distribute, all copyright, patent, trademark, and
          attribution notices from the Source form of the Work,
```

```
        excluding those notices that do not pertain to any part of
        the Derivative Works; and

    (d) If the Work includes a "NOTICE" text file as part of its
        distribution, then any Derivative Works that You distribute must
        include a readable copy of the attribution notices contained
        within such NOTICE file, excluding those notices that do not
        pertain to any part of the Derivative Works, in at least one
        of the following places: within a NOTICE text file distributed
        as part of the Derivative Works; within the Source form or
        documentation, if provided along with the Derivative Works; or,
        within a display generated by the Derivative Works, if and
        wherever such third-party notices normally appear. The contents
        of the NOTICE file are for informational purposes only and
        do not modify the License. You may add Your own attribution
        notices within Derivative Works that You distribute, alongside
        or as an addendum to the NOTICE text from the Work, provided
        that such additional attribution notices cannot be construed
        as modifying the License.

    You may add Your own copyright statement to Your modifications and
    may provide additional or different license terms and conditions
    for use, reproduction, or distribution of Your modifications, or
    for any such Derivative Works as a whole, provided Your use,
    reproduction, and distribution of the Work otherwise complies with
    the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
    any Contribution intentionally submitted for inclusion in the Work
    by You to the Licensor shall be under the terms and conditions of
    this License, without any additional terms or conditions.
    Notwithstanding the above, nothing herein shall supersede or modify
    the terms of any separate license agreement you may have executed
    with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade
    names, trademarks, service marks, or product names of the Licensor,
    except as required for reasonable and customary use in describing the
    origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or
    agreed to in writing, Licensor provides the Work (and each
    Contributor provides its Contributions) on an "AS IS" BASIS,
    WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
    implied, including, without limitation, any warranties or conditions
    of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
    PARTICULAR PURPOSE. You are solely responsible for determining the
    appropriateness of using or redistributing the Work and assume any
    risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory,
    whether in tort (including negligence), contract, or otherwise,
    unless required by applicable law (such as deliberate and grossly
    negligent acts) or agreed to in writing, shall any Contributor be
    liable to You for damages, including any direct, indirect, special,
    incidental, or consequential damages of any character arising as a
    result of this License or out of the use or inability to use the
    Work (including but not limited to damages for loss of goodwill,
    work stoppage, computer failure or malfunction, or any and all
    other commercial damages or losses), even if such Contributor
    has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
    the Work or Derivative Works thereof, You may choose to offer,
    and charge a fee for, acceptance of support, warranty, indemnity,
    or other liability obligations and/or rights consistent with this
    License. However, in accepting such obligations, You may act only
    on Your own behalf and on Your sole responsibility, not on behalf
    of any other Contributor, and only if You agree to indemnify,
    defend, and hold each Contributor harmless for any liability
    incurred by, or claims asserted against, such Contributor by reason
    of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS
```

```
APPENDIX: How to apply the Apache License to your work.

   To apply the Apache License to your work, attach the following
   boilerplate notice, with the fields enclosed by brackets "[]"
   replaced with your own identifying information. (Don't include
   the brackets!)  The text should be enclosed in the appropriate
   comment syntax for the file format. We also recommend that a
   file or class name and description of purpose be included on the
   same "printed page" as the copyright notice for easier
   identification within third-party archives.

Copyright 2014 Open Source Robotics Foundation, Inc.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.
```